

Programación para Física y Astronomía

Departamento de Física.

Coordinadora: C Loyola

Profesores C Femenías / C Ruiz / D García / S villanova

Primer Semestre 2026

Universidad Andrés Bello

Departamento de Física y Astronomía



Resumen - Semana 8, Sesión 1 (Sesión 15)

Introducción y Contexto

Fundamentos de Pandas para Análisis de Datos

Fundamentos de Pandas

Limpieza de datos

Lectura y Exportación de Datos

Visualización de Datos con Pandas

Ejercicios de practica

Introducción y Contexto

Introducción y Contexto

Repaso de la Semana 7

- **Semana 7, Sesión 1 (Sesión 13):**
 - Exploramos gráficos avanzados en Matplotlib (subplots múltiples, histogramas, 3D).
 - Tuvimos un vistazo opcional a pandas para el manejo de datos tabulares.
- **Semana 7, Sesión 2 (Sesión 14):**
 - Realizamos una **sesión de ejercicios** en la que se implementaron arreglos con NumPy y su posterior visualización usando librerías como Matplotlib.
 - Aclaremos dudas tras la entrega.
- **Objetivo de hoy:** Iniciar **programación orientada a análisis de datos** en Python y repasar análisis básico de datos en NumPy/pandas.

Introducción y Contexto

Objetivos de la Sesión 15

- **Explorar** el uso de **pandas** para análisis y manipulación de datos.
- **Aplicar** operaciones básicas y avanzadas de pandas en contextos físicos y astronómicos.

Fundamentos de Pandas para Análisis de Datos

¿Por qué Pandas para Análisis de Datos?

- **DataFrames:** estructuras tabulares similares a Excel/hojas de cálculo
- **Lectura:** archivos CSV, Excel, JSON automáticamente
- **Operaciones:** filtrado, agrupación, estadísticas descriptivas
- **Limpieza:** manejo de datos faltantes, duplicados
- **Integración:** funciona perfectamente con NumPy y Matplotlib

Para Física/Astronomía

Ideal para analizar datos experimentales, observaciones astronómicas, resultados de simulaciones.

¿Qué es pandas?

- **pandas** es una biblioteca de Python para análisis y manipulación eficiente de datos.
- Permite trabajar con datos tabulares (tipo Excel) de forma sencilla y poderosa.
- Ofrece estructuras clave: **Series** (vectores) y **DataFrame** (tablas).
- Esencial para ciencia de datos, física computacional y astronomía.

Ventajas

- Lectura y escritura de archivos (CSV, Excel, etc.)
- Filtrado, selección y limpieza de datos
- Estadísticas y agrupaciones rápidas
- Integración con NumPy y Matplotlib

Fundamentos de Pandas

Fundamentos de Pandas

Series: Vector unidimensional

```
1 import pandas as pd
2
3 # Series
4 s = pd.Series([1.5, 2.0, 0.8])
5
6 s
```

# 0	
0	1.5
1	2.0
2	0.8

- **Series:** vector unidimensional con etiquetas de índice.

```
1 import pandas as pd
2
3 # Series
4 s = pd.Series([1.5, 2.0, 0.8], index=['objeto1',
   ↪  'objeto2', 'objeto3'])
5
6 s
```

# 0	
objeto1	1.5
objeto2	2.0
objeto3	0.8

Discusión: Una **Series** es como un vector con índice.

- **DataFrame**: tabla bidimensional con filas y columnas etiquetadas.

```
1 import pandas as pd
2
3 # DataFrame
4 df = pd.DataFrame({
5     'masa': [1.5, 2.0, 0.8],
6     'velocidad': [10.5, 8.2, 15.3]
7 })
8
9 df
```

	# masa	# velocidad
0	1.5	10.5
1	2.0	8.2
2	0.8	15.3

Discusión: Un DataFrame es como una tabla de Excel.

- **DataFrame**: tabla bidimensional con filas y columnas etiquetadas.

```
1 df.rename(index={3: "Fila 3"}, inplace=True)
2
3 df
```

	# masa	# velocidad
1	1.5	10.5
2	2.0	8.2
Fila 3	0.8	15.3

Discusión: Se pueden personalizar las etiquetas de filas y columnas.

```
1 # Acceso por columna
2 df['masa']
```

# masa	
0	1.5
1	2.0
2	0.8

Discusión: Este ejemplo muestra cómo acceder a una columna específica de un DataFrame utilizando su nombre como clave. Es útil para realizar operaciones o análisis sobre una sola variable.

```
1 # Acceso por fila (iloc: por posición, loc: por
  ↳ etiqueta)
2 df.iloc[2] # Tercera fila
3 df.loc["Fila 3"] # Fila con nombre "Fila 3"
```

# Fila 3	
masa	0.8
velocidad	15.3

Discusión: Aquí se muestra cómo acceder a filas específicas de un DataFrame, ya sea por su posición numérica ('iloc') o por su etiqueta ('loc'). Esto es útil para inspeccionar o modificar datos en filas específicas.

Fundamentos de Pandas

Operaciones vectorizadas y estadísticas

```
1 # Operaciones sobre columnas
2 df['energia_cinetica'] = 0.5 * df['masa'] * df['velocidad']**2
3
4 df
```

	# masa	# velocidad	# energia_cinetica
1	1.5	10.5	82.6875
2	2.0	8.2	67.24
Fila 3	0.8	15.3	93.63600000000002

Discusión: Las operaciones se aplican a toda la columna sin necesidad de bucles explícitos.

Fundamentos de Pandas

Estadísticas descriptivas rápidas

```
1 # Operaciones sobre columnas
2 # Estadísticas rápidas
3 df['energia_cinetica'].mean() # Promedio de energía cinética
4 # Output: 81.18783333333334
5 df.describe()
```

	# masa	# velocidad	# energia_cinetica
count	3.0	3.0	3.0
mean	1.4333333333333333	11.333333333333334	81.18783333333334
std	0.6027713773341707	3.6226141573915016	13.261747776342819
min	0.8	8.2	67.24
25%	1.15	9.35	74.96375
50%	1.5	10.5	82.6875
75%	1.75	12.9	88.16175000000001
max	2.0	15.3	93.63600000000002

Discusión: `df.describe()` entrega un diagnóstico rápido del comportamiento general de los datos.

Fundamentos de Pandas

Agrupación y resumen de datos

```
1 df = pd.DataFrame({
2     'masa': [1.5, 2.0, 0.8, 1.2],
3     'velocidad': [10.5, 8.2, 15.3, 11.1],
4     'temperatura': [250, 300, 390, 280]
5 })
6
7 # Crear una columna categórica
8 df['categoria'] = pd.cut(df['temperatura'], bins=[0,273,373,1000],
9     ↪ labels=['Sólido', 'Líquido', 'Gaseoso'])
10 df
```

	# masa	# velocidad	# energia_cinetica	📦 categoria	
1		1.5	Missing value	82.6875	Alta
2		2.0	8.2	67.24	Baja
Fila 3		0.8	15.3	93.63600000000002	Alta

Discusión: Agrupar datos permite obtener estadísticas por categorías físicas (masas, temperaturas, etc.).

Fundamentos de Pandas

Agrupación y resumen de datos

```
1 # Agrupar y calcular estadísticas
2 df.groupby('categoria')['temperatura'].mean() # Puede ser .mean(), .std(), .sum(), etc.
```

categoria	# temperatura
Solido	127.55737704918033
Liquido	330.65625
Gaseoso	389.0

Discusión: Agrupar datos permite obtener estadísticas por categorías físicas (masas, temperaturas, etc.).

```
1 # Operaciones básicas
2 df.info()           # Información del
   ↪ DataFrame
```

Output:

```
<class 'pandas.core.frame.DataFrame'>
```

```
RangeIndex: 4 entries, 0 to 3
```

```
Data columns (total 4 columns):
```

#	Column	Non-Null Count	Dtype
0	masa	4 non-null	float64
1	velocidad	4 non-null	float64
2	temperatura	4 non-null	int64
3	categoría	4 non-null	category

```
dtypes: category(1), float64(2), int64(1)
```

```
memory usage: 468.0 bytes
```

```
1 # Operaciones básicas
2 df.shape           # Dimensiones
   ↪ (filas, columnas)
```

Output:

```
(4, 4)
```

Discusión: Estas operaciones básicas permiten inspeccionar rápidamente la estructura y dimensiones de un DataFrame, facilitando el análisis inicial de los datos.

Fundamentos de Pandas

Selección de Columnas importantes

```
1 # Seleccionar columnas
2 subset = df[['masa', 'velocidad']] # Múltiples columnas
3 subset
```

	# masa	# velocidad
0	1.1622909903858079	6.294779128285901
1	1.2575392838565098	5.195686868016088
2	1.5875412989737128	12.157919722098546
3	1.2514011420766056	13.672540022650423
4	1.1538870112094566	16.95447565316892
5	1.4731483172445072	13.015316641086615
6	0.9072892094622428	12.942605710820246
7	0.74477645177605	18.15180828289533
8	1.4300533214655435	13.98126996753548
9	0.686840808473431	6.4791834216107

Discusión: La selección de columnas específicas permite centrarse en variables de interés, facilitando el análisis y la visualización de datos relevantes.

Fundamentos de Pandas

Otra forma de Selección y Filtrado de Datos

```
1 # Filtrado con condiciones
2 altas_temp = df[df['temperatura'] > 300]
3 altas_temp
```

# velocidad	# temperatura	📦 categoria
13.015316641086615	330	Liquido
7.80473808300243	368	Liquido
16.036247391526743	363	Liquido
14.312656446831161	382	Gaseoso
14.2988753285948	389	Gaseoso
19.257432372696933	306	Liquido
15.414132414861374	345	Liquido
5.531101211711803	362	Liquido
10.711592605859591	386	Gaseoso
16.888192119760735	363	Liquido

Discusión: El filtrado de datos permite identificar subconjuntos relevantes, como aquellos con temperaturas extremas, facilitando un análisis más enfocado.

Fundamentos de Pandas

Otra forma de Selección y Filtrado de Datos

```
1 # Filtrado múltiple
2 criteria = (df['masa'] > 1.0) & (df['velocidad'] > 10) & (df['categoria'] == 'Liquido')
3 filtered_df = df[criteria]
4 filtered_df
```

#	masa	velocidad	temperatura	categoria
3	1.2514011420766056	13.672540022650423	284	Liquido
5	1.4731483172445072	13.015316641086615	330	Liquido
23	1.0930179125964166	18.015052688414393	295	Liquido
28	1.910202686031392	19.257432372696933	306	Liquido
49	1.0594663777168525	16.735615888787272	279	Liquido
64	1.473022133491637	12.222812720898194	353	Liquido
65	1.315947212724838	13.011127133650767	283	Liquido
69	1.2258166445419816	14.969849494919098	327	Liquido
81	1.800160284661923	19.9797030843529	277	Liquido
87	1.2158278235581306	17.6616430681612	331	Liquido

Discusión: El filtrado múltiple permite combinar varias condiciones para extraer subconjuntos específicos de datos, facilitando análisis detallados basados en múltiples criterios.

Limpieza de datos

Limpieza de datos

Detectar Valores Faltantes

```
1 import numpy as np
2
3 # Simular algunos valores faltantes para inspección
4 df.loc[[1, 3], 'temperatura'] = np.nan
5
6 # Detectar valores faltantes
7 print("Valores nulos por columna:")
8 df.isnull().sum()
```

# 0	
masa	0
velocidad	0
temperatura	50
categoría	0

Discusión: Identificar valores faltantes es crucial para decidir cómo manejarlos en el análisis de datos.

Limpieza de datos

Manejo de Datos Faltantes y Limpieza

```
1 # Opciones para manejar datos faltantes
2 df_dropped = df.dropna() # Eliminar filas con NaN
3 df_dropped
```

	# masa	# velocidad	# temperatura	categoria
0	0.702945440225168	6.3865191775676955	109.0	Solido
1	0.9690671393777127	6.048542373822548	23.0	Solido
2	1.333310990489152	15.240299023778565	234.0	Solido
3	1.9231138793475124	19.90359211556003	353.0	Liquido
6	1.105603194347084	11.443547055237433	220.0	Solido
7	1.95634581987672	10.806381878743224	352.0	Liquido
10	1.3044770518799047	11.894737405806785	390.0	Gaseoso
14	1.3147638161813286	17.159520563824376	311.0	Liquido
17	0.6504724438774485	18.12720739180797	340.0	Liquido
18	1.1617941760020265	8.821518400931424	39.0	Solido

50 rows x 4 cols 10 per page Page 1 of 5

Discusión: Eliminar filas con datos faltantes es una estrategia simple pero puede resultar en pérdida de información valiosa.

Limpieza de datos

Manejo de Datos Faltantes y Limpieza

```
1 # Opciones para manejar datos faltantes
2 numeric_means = df.select_dtypes(include=[np.number]).mean()
3 numeric_means
```

# 0	
masa	1.2946635181576476
velocidad	12.879820643921464
temperatura	230.5

Discusión: Calculamos las medias solo de las columnas numéricas para evitar errores al rellenar.

```
1 # Rellenar solo columnas numéricas con sus medias
2 df_filled = df.fillna(numeric_means)
3 df_filled
```

	# masa	# velocidad	# temperatura	categoria
0	0.702945440225168	6.3865191775676955	109.0	Solido
1	0.9690671393777127	6.048542373822548	230.5	Solido
2	1.333310990489152	15.240299023778565	234.0	Solido
3	1.9231138793475124	19.90359211556003	230.5	Liquido
4	0.9042465857442007	12.41097460994106	230.5	Solido

Discusión: Conviene mostrar solo la estrategia correcta: rellenar columnas numéricas sin afectar variables categóricas

Lectura y Exportación de Datos

Lectura y Exportación de Datos

Lectura de Archivos y Exportación

Lectura de archivos: pandas facilita la importación de datos desde diversos formatos comunes en física y astronomía, como CSV y Excel.

```
1 # Leer diferentes formatos Ejemplos
2 df_csv = pd.read_csv('datos_experimento.csv') # CSV por defecto
3 df_excel = pd.read_excel('mediciones.xlsx', sheet_name='Hoja1') # Excel
4
5 # Parámetros útiles para CSV
6 df_custom = pd.read_csv('datos.csv',
7                          sep=';',          # Separador
8                          decimal=',',     # Separador decimal
9                          encoding='utf-8', # Codificación
10                         skiprows=2)       # Saltar filas
11
12 # Exportar resultados
13 df.to_csv('resultados_procesados.csv', index=False)
14 df.to_excel('analisis.xlsx', sheet_name='Datos')
15
16 # Exportar solo columnas específicas
17 df[['masa', 'velocidad']].to_csv('energia_datos.csv')
```

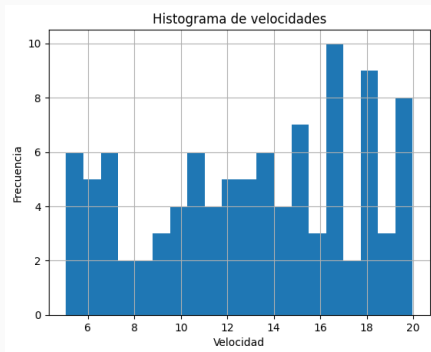
Visualización de Datos con Pandas



Visualización de Datos con Pandas

Visualización rápida con pandas

```
1 import matplotlib.pyplot as plt
2
3 df['velocidad'].hist(bins=20)
4 plt.title('Histograma de velocidades')
5 plt.xlabel('Velocidad')
6 plt.ylabel('Frecuencia')
7 plt.show()
```

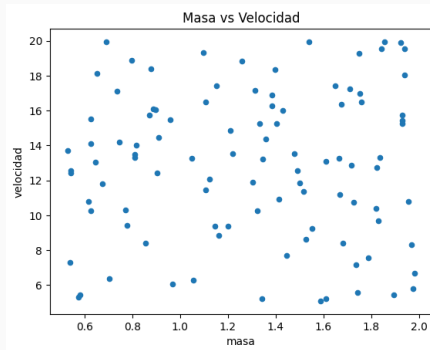


Discusión: pandas integra métodos de visualización para análisis

Visualización de Datos con Pandas

Visualización Integrada con Pandas y Matplotlib

```
1 import matplotlib.pyplot as plt
2 # Pandas tiene métodos de plotting integrados
3 df.plot(x='masa', y='velocidad', kind='scatter', title='Masa vs Velocidad')
4 plt.show()
```



Discusión: Este gráfico muestra la relación entre masa y velocidad en un scatter plot, útil para identificar patrones.

Ejercicios de practica

Ejercicios de practica

Dataset base: Catálogo simple de estrellas cercanas

```
1 import numpy as np
2 import pandas as pd
3
4 df = pd.DataFrame({
5     "estrella": ["Sol", "Alpha Cen A", "Proxima Cen", "Sirius A",
6                 "Tau Ceti", "Vega", "Barnard", "61 Cyg A"],
7     "temperatura_K": [5778, 5790, 3042, 9940, 5344, 9602, 3134, 4450],
8     "radio_Rsol": [1.00, 1.22, 0.15, 1.71, 0.79, 2.36, 0.20, 0.67],
9     "distancia_pc": [0.00, 1.34, 1.30, 2.64, 3.65, 7.68, 1.83, 3.48],
10    "magnitud_V": [-26.7, 0.01, 11.13, -1.46, 3.50, 0.03, 9.54, np.nan],
11    "categoria": ["Solar", "Solar", "Enana roja", "Caliente",
12                 "Solar", "Caliente", "Enana roja", "Enana naranja"]
13 })
14
15 df
```

Nota

Se usa un dataset pequeño y legible para concentrarnos en pandas: inspección, filtrado, limpieza, agrupación y exportación.



Enunciado

- Visualiza → `head()`, `tail()` y `df.columns`.
- Reporta unidades de las variables numéricas principales.

Conceptos: Inspección inicial. **Astronomía:** Revisión de magnitudes y unidades.



```
1 # Inspección inicial del DataFrame
2 df.head()
3 df.tail()
4 df.columns.tolist()
5
6 # Unidades:
7 # temperatura_K [K], radio_Rsol [R_sun],
8 # distancia_pc [pc], magnitud_V [mag]
```

Discusión: Validamos estructura y unidades para evitar errores de interpretación.



Enunciado

- Ejecuta → `df.shape`, `df.info()` y `df.describe()`.
- Identifica columnas numéricas vs no numéricas.

Conceptos: Estructura y estadísticas rápidas.

Ejercicios de practica

Solución 2 de Referencia:

Dimensiones, tipos y describir



```
1 # Dimensiones, tipos y estadísticas
2 print("shape:", df.shape)
3 print("="*40)
4 df.info()
5 print("="*40)
6 print(df.describe())
7
8 # Columnas numéricas vs no numéricas
9 num_cols = df.select_dtypes(include='number').columns.tolist()
10 non_num_cols = df.select_dtypes(exclude='number').columns.tolist()
11 print("Numéricas:", num_cols)
12 print("No numéricas:", non_num_cols)
```

Discusión: Diferenciar tipos guía la limpieza y transformaciones posteriores.



Enunciado

- Crea un subconjunto con las columnas `estrella`, `temperatura_K` y `distancia_pc`.
- Muestra ese subconjunto.

Conceptos: Selección de columnas.

Solución 3 de Referencia: Selección de columnas



```
1 # Seleccionar columnas de interés
2 subset = df[['estrella', 'temperatura_K', 'distancia_pc']]
3 subset
```

Discusión: Seleccionar columnas ayuda a enfocarse en las variables relevantes antes de analizar.



Enunciado

- Filtra estrellas con `temperatura_K > 5000` y `distancia_pc < 5`.
- Cuenta cuántas filas cumplen la condición y muestra las columnas `estrella`, `temperatura_K` y `distancia_pc`.

Conceptos: Máscaras booleanas y filtrado.

Solución 4 de Referencia: Filtrado de estrellas



```
1 # Filtrado con dos condiciones
2 filtro = (df['temperatura_K'] > 5000) & (df['distancia_pc'] < 5)
3 cercanas_calidas = df[filtro]
4
5 print("Cantidad:", len(cercanas_calidas))
6 cercanas_calidas[['estrella', 'temperatura_K', 'distancia_pc']]
```

Discusión: Este tipo de filtro es de los usos más comunes de pandas en análisis inicial.



Enunciado

- Crea la columna `luminosidad_rel` usando la relación:
- `df['luminosidad_rel'] = df['radio_Rsol']**2 * (df['temperatura_K']/5778)**4`
- Muestra `estrella` y `luminosidad_rel`.

Conceptos: Crear columnas con operaciones vectorizadas.

Solución 5 de Referencia: Crear una columna derivada



```
1 # Luminosidad relativa aproximada
2 df['luminosidad_rel'] = df['radio_Rsol']**2 * (df['temperatura_K'] / 5778)**4
3 df[['estrella', 'luminosidad_rel']]
```

Discusión: Agregar columnas nuevas es una de las operaciones más útiles para enriquecer un dataset.



Enunciado

- Detecta valores faltantes con `df.isnull().sum()`.
- Rellena `magnitud_V` con el promedio de esa columna y guarda el resultado en `df_filled`.

Conceptos: Detección y limpieza básica de nulos.



```
1 # Detectar valores faltantes
2 print(df.isnull().sum())
3
4 # Rellenar magnitud con el promedio
5 mag_media = df['magnitud_V'].mean()
6 df_filled = df.fillna({'magnitud_V': mag_media})
7 df_filled[['estrella', 'magnitud_V']]
```

Discusión: Primero se detecta el problema y luego se aplica una solución simple y explícita.



Enunciado

- Agrupa por `categoría`.
- Calcula la media de `temperatura_K` y `distancia_pc` para cada grupo.

Conceptos: `groupby` y medias por grupo.



```
1 resumen = df.groupby('categoria')[['temperatura_K', 'distancia_pc']].mean()  
2 print(resumen)
```

Discusión: `groupby( ).mean( )` resume información de manera clara con muy pocas líneas.



Enunciado

- Dibuja un histograma de `temperatura_K`.
- Exporta `df_filled` a un archivo CSV llamado `catalogo_estrellas.csv`.

Conceptos: visualización rápida y persistencia con `to_csv`.

Ejercicios de practica

Solución 8 de Referencia:

Visualización y exportación



```
1 import matplotlib.pyplot as plt
2
3 # Histograma de temperaturas
4 df['temperatura_K'].hist(bins=6)
5 plt.xlabel('Temperatura [K]')
6 plt.ylabel('Frecuencia')
7 plt.title('Distribución de temperaturas')
8 plt.show()
```

```
1 # Si aún no existe df_filled, se crea rellenando magnitud_V
2 mag_media = df['magnitud_V'].mean()
3 df_filled = df.fillna({'magnitud_V': mag_media})
4
5 # Exportar CSV
6 df_filled.to_csv('catalogo_estrellas.csv', index=False)
```

Discusión: Se combina una visualización rápida con la exportación de una versión limpia del dataset.